Planning at the Speed of Control

Fusing a JEPA world-model planner into one GPU kernel: an equal-hardware optimisation ladder for LeWorldModel on H100, from exact caching and iCEM to a ThunderKittens megakernel

Antoine Nguyen · **24433599**
Research presentation · [Affiliation]
Supervisor: [supervisor name] · October 2026

The project in one slide

- JEPA world models plan by searching over action sequences in latent space; LeWM is the smallest of them, and its released planner still takes 622 ms per plan for one robot on an H100.
- Question: how much of that latency can be removed without changing the model, the planner or its task success, and which kinds of change account for it?
- Method: a seven-rung ladder on one H100, one change per rung, each measured end to end at batch 1 and in closed loop on the same 300 Push-T episodes.
- Answer: exact caching gives 1.65x; iCEM is worth nothing in eager PyTorch and 1.38x in a megakernel; one persistent kernel per plan gives 46.5 ms, with success unchanged.

622 ms
reference LeWM, CEM, FP32

182 ms
best PyTorch path (LeWM-Flash)

46.5 ms
one megakernel launch per plan

–0.3 pts
success vs reference, 300 paired episodes

One H100, batch 1, end-to-end plan latency (median); 300 paired episodes

Why this topic: a latency problem

- A controller replans every few steps and must answer within the period it plans for: a 10 Hz loop has 100 ms for sensing, planning and actuation together.
- LeWM's model is tiny: a cost call for 300 candidates is under 0.1 ms of tensor-core arithmetic, and all its weights fit in the H100's L2 cache.
- Its plan is slow for other reasons: re-encoding inside every cost call, a host round trip per optimiser iteration, every operator launched from Python, and a chain of 900 dependent layer evaluations per plan.
- Neither FLOPs nor memory bandwidth binds. The latency of each link of the chain does.

15M
parameters in the whole model

2,820
rollouts per plan (iCEM; CEM: 9,000)

900
dependent layer evaluations per plan

20 ms
one eager cost call, at 60 or 300 candidates

The released planner is 30 iterations x 5 predictor passes x 6 layers, each link a handful of small kernels. Batch 1 is what a robot runs.

How I got here (reflection)

1. **LeWM-Flash on an L40S**
   A ledger of PyTorch-level changes to the released planner, each run as a paired 300-episode sweep: caching, no host syncs, compilation, TF32, iCEM and BF16 accepted; warm starts, whole-loop compile, CEM-GD and static BF16 rejected.

2. **The wall**
   The best PyTorch planner still spent most of its time between kernels: 82 ms per solve on an H100 for a few milliseconds of arithmetic. Compilers fuse operators, not an optimiser loop.

3. **The megakernel**
   Rewrite the whole iCEM solve as one persistent ThunderKittens kernel on Hopper, following the LLM-decoding megakernels: 76.9 ms for the first correct kernel, 41.3 ms after five tuning steps.

4. **The ladder**
   Measure every step on the same H100 against the reference, at batch 1 end to end, with paired closed-loop success; then fuse the encoder so that a plan is one launch.

Literature review: three lineages, one gap

JEPA world models

I-JEPA, V-JEPA, DINO-WM, PLDM, V-JEPA 2-AC, LeWM: all plan with CEM-family MPC in latent space. Latency is reported coarsely, if at all (53 s per DINO-WM call, 16 s per V-JEPA 2-AC action, LeWM 'under one second'), and every speed-up changes the model or the planner.

CEM to iCEM

CEM (Rubinstein) and PETS/PlaNet made latent CEM-MPC the standard. iCEM adds a decaying population and elite memory: 2.7-22x fewer samples. CEM-GD shows the catch: fewer samples do not mean less time unless the per-iteration cost falls too.

Kernels and fusion

Small-batch inference is launch- and latency-bound. FlashAttention fuses an operator; torch.compile and CUDA graphs fuse regions; megakernels (Hazy Llama-1B, FlashFormer, MPK) fuse a whole LLM forward pass into one launch. None fuses an optimiser loop.

The gap. No work holds a JEPA world model and its planner fixed and measures what the execution of the planner costs; iCEM has run on LeWM only as an evaluated solver; and no kernel fuses a whole sampling-based planning loop with a neural encoder, a transformer rollout, elite selection and distribution updates. GPU MPC (STORM, cuRobo, MPPI-Generic) is fast only with analytic dynamics.

Research questions

RQ1 How much of the reference planner's latency is redundant work that an exact change removes?

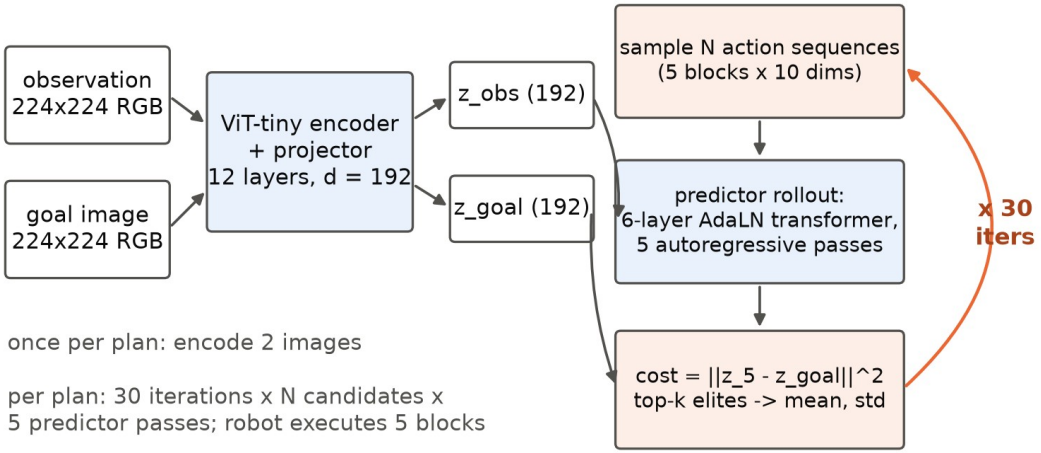
RQ2 Does iCEM make planning faster, by how much, and does it change task success?

RQ3 How much does executing the whole planner as one persistent kernel gain over the best PyTorch implementation of the same planner?

RQ4 Do the megakernel's numerics and the fused encoder preserve accuracy, against FP32 on identical inputs and in closed loop?

One change per rung, one H100, one container image; primary metric: batch-1 end-to-end plan latency.

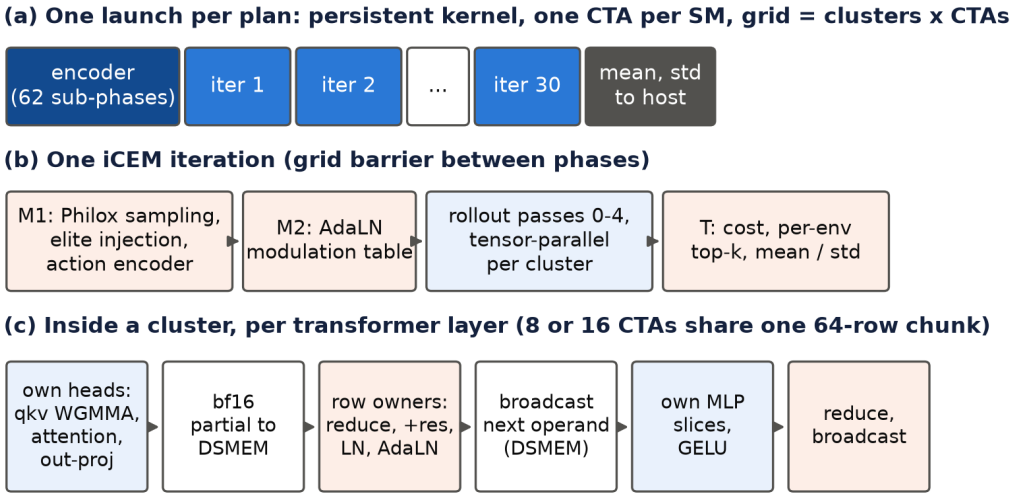
Method (1): the system and the ladder



- R0 reference: CEM 300 x 30, FP32, re-encodes in every cost call.
- R1 + encode once per plan, no host sync (exact).
- R2 + bucketed iCEM: 300 -> 60 candidates, 5 retained elites (planner).
- R2b LeWM-Flash: + compile, TF32, BF16 (best PyTorch).
- R3 megakernel: predictor + all 30 iterations in one launch.
- R4 + ViT encoder fused: one launch per plan.
- Side rung R3-CEM: the megakernel running plain CEM.

LeWM on Push-T: ViT-tiny encoder, 6-layer AdaLN predictor, 5-step rollout, latent goal cost.

Method (2): one kernel per plan



Weights: 1,472 pre-swizzled 24 KB bf16 panels. Each warpgroup's thread 0 refills its 2-slot shared-memory ring with cp.async.bulk from a static schedule, so weight prefetch runs through op, layer, pass and barrier boundaries.

- Persistent grid: one CTA per SM, clusters of 8 or 16, grid barriers between iCEM phases; sampling (Philox), top-k and elite updates in the kernel.
- Cluster tensor parallelism: each CTA owns heads and MLP slices; row owners reduce and broadcast over distributed shared memory.
- Weights as a static stream: pre-swizzled 24 KB panels, cp.async.bulk into a self-refilling ring, no producer warp (keeps 255 registers).
- Exact KV reuse; BF16 operands with FP32 accumulation, residual and costs.

Method (3): measuring what a robot would see

Batch-1 latency

The whole replanning call (preprocess, encode, solve, action to host) with CUDA events; 61 one-robot episodes per rung, first dropped; 72-76 plans; median with bootstrap CI and p90.

300 paired episodes

Push-T seeds 0-2 x 100 episodes; start states fixed by seed, so every rung sees the same episodes. Wilson intervals, exact McNemar, paired bootstrap of the difference.

Numerical acceptance

Before any closed loop: cost error and top-30 overlap against FP32 on identical candidates; the bar is the accepted BF16 autocast path. Encoder: embedding error and cost error.

What 300 pairs can see. At the observed 12-13% discordance, a paired difference of about 6 points is detectable at 80% power; 2 points would need about 2,400 pairs. Every accuracy claim in this work is a claim of no detectable change at that resolution.

The plan, and how it ran

Phase 0

LeWM-Flash (L40S)

Paired 300-episode sweeps of PyTorch-level changes. Accepted: caching, no syncs, compile, TF32, iCEM, BF16. Rejected: warm starts (success fell to 34-42%), whole-loop compile, CEM-GD, static BF16.

Phase 1

Megakernel (H100)

Kernel built against FP32 and BF16 references; five tuning steps, 76.9 -> 41.3 ms solve; 300-episode runs of the reference, LeWM-Flash and the kernel on the same GPU.

Phase A

First ladder pass

Batch-1 latency of R0-R3 and R3-CEM; quality runs of R1 and R2.

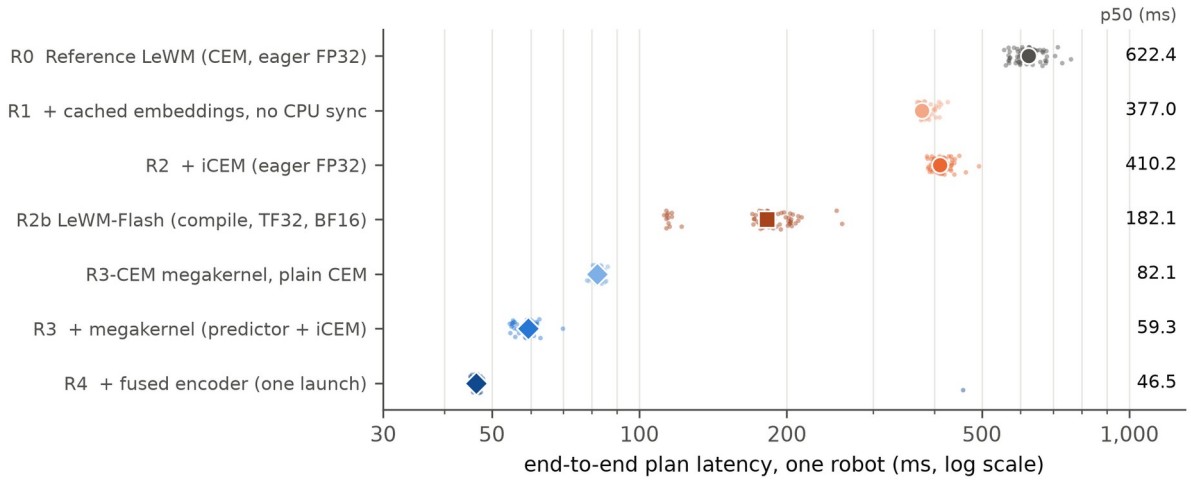
Final

Fused encoder + ladder

Encoder in the kernel, parity checked; second batch-1 pass of all seven rungs; quality runs of R3, R3-CEM and R4.

Compute: about $31.50 of H100 time against a $35 budget (extended from $10 when the ladder was added). Each batch-1 pass cost $10-12 because the harness rebuilds the simulator and model for every seed.

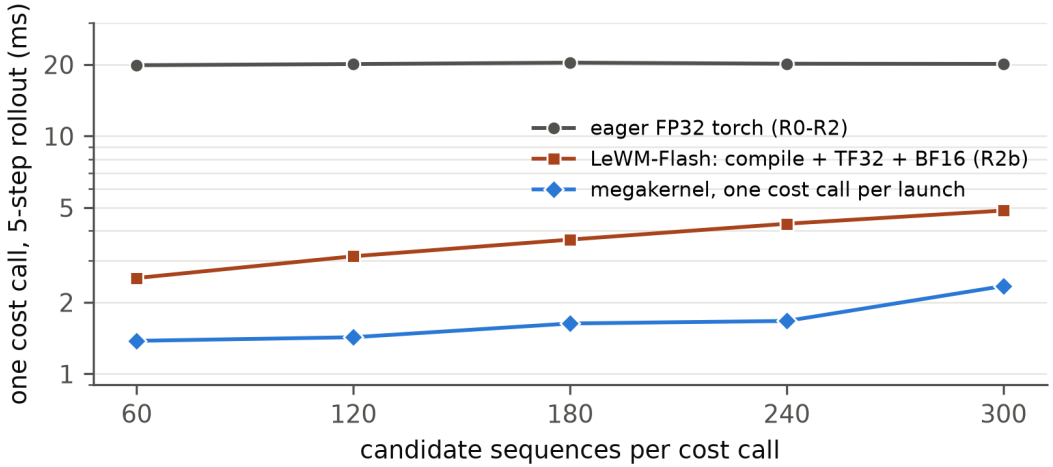
Result 1: 622 ms to 46.5 ms on the same GPU



- Caching: 1.65x, exact.
- iCEM in eager PyTorch: 0.92x.
- LeWM-Flash: 3.4x over the reference.
- Megakernel: 6.9x over eager iCEM, 3.1x over LeWM-Flash.
- Fused encoder: 1.28x more; 13.4x in total, p90 47.0 ms.
- Megakernel rungs are tight: R4's interquartile range is 46.1-46.8 ms.

Batch-1 end-to-end plan latency, log scale; small points are individual plans, markers are medians.

Result 2: iCEM pays only when execution does



One cost call (5-step rollout of every candidate) against the number of candidates, H100.

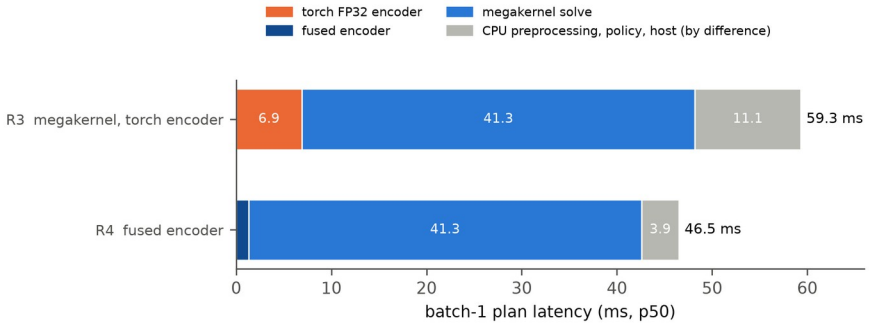
A sample-efficient planner is only as fast as its smallest batch. Measured only in eager PyTorch, iCEM would have been rejected; measured only on the kernel, it would look a third as useful as its evaluation count suggests.

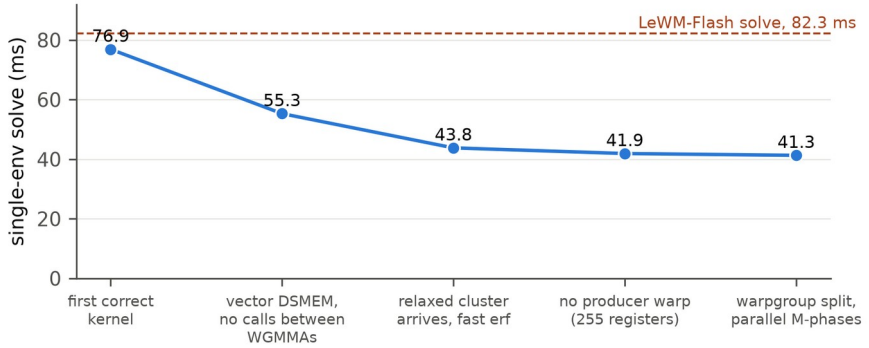
0.92x
iCEM vs CEM, eager PyTorch

1.38x
iCEM vs CEM, inside the kernel

- An eager cost call costs 20 ms at 60 candidates and at 300: it is a few hundred launches, not arithmetic.
- Inside the kernel the call scales with the population, so iCEM's 3.19x fewer rollouts become 1.38x (R3-CEM 82.1 -> R3 59.3 ms).
- Success unchanged both ways (265 vs 263; 262 vs 261 of 300).

Result 3: where the time goes, and how it fell





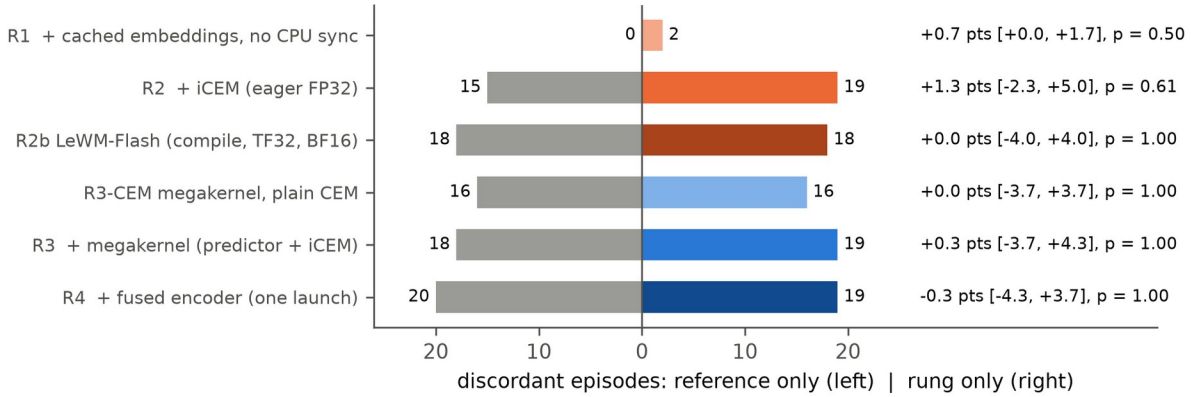
41.3 ms
kernel solve
(Flash: 82.3)

1.3 ms
fused encoder
(FP32 torch: 6.9)

0.65 µs
cluster barrier
(was 3.6 µs)

- Each tuning step removed latency, not arithmetic: no calls between WGMMAs, relaxed cluster arrives, fast erf, no producer warp.
- Every attempt that raised register pressure (peeling, K/V prefetch, owner prefetch) made the kernel slower.
- What remains is a 900-layer dependent chain; going far below 40 ms needs fewer cross-SM hand-offs per layer, a redesign.

Result 4: task success does not move



Grey: episodes only the reference solved; coloured: episodes only the rung solved. Annotation: paired difference, bootstrap 95% CI, exact McNemar p.

- Every rung: 260-265 of 300; no paired test near significance (smallest p = 0.50).
- Whole ladder, R4 vs R0: 19 won / 20 lost, -0.3 points [-4.3, +3.7].
- The exact rung differs in 2 episodes; any new random stream in 32-39: the size of sampling noise.
- Kernel cost error vs FP32 matches BF16 autocast (max rel. 0.0005-0.0043); fused encoder closer to FP32 than autocast.

Conclusions, and what I cannot claim

RQ1 About 40% of the reference latency is redundant work: caching and no host syncs, 622 -> 377 ms, exact.

RQ2 It depends on execution: 0.92x in eager PyTorch, 1.38x in the megakernel; success unchanged either way.

RQ3 3.1x over the best PyTorch path end to end (2.0x on the solve), 6.9x over eager iCEM, at the same numerics class.

RQ4 Yes, within resolution: cost error matches BF16 autocast; -0.3 points [-4.3, +3.7] on 300 paired episodes.

Issues and limits

- 300 pairs resolve about 6 points of success, no less.
- PyTorch rungs moved up to 18% between two passes; the kernel rungs under 2%.
- One model, one task, one GPU generation; Hopper-only kernel.
- Simulation only; only the planning call was optimised.
- R2b is bimodal and R4 has one 458 ms outlier, causes not isolated.
- Budget: $31.50 of $35, two latency passes, one quality run per rung.

Future work: towards a research programme

1. **Spend the saved latency on planning**
   Sweep iterations, population and elites at a few ms per plan: map the latency-success frontier, not one point.

2. **Fewer hand-offs per layer**
   A single-barrier all-reduce, or one layer per cluster with resident (FP8) weights.

3. **Compose with model-level speed-ups**
   Fast-LeWM's parallel prefix prediction would cut the chain from 5 passes per iteration to 1.

4. **Other world models, other GPUs**
   DINO-WM and V-JEPA 2-AC push back towards throughput; Blackwell and Jetson Thor change the primitives; automate with megakernel compilers.

5. **Evaluate at deployment resolution**
   About 2,400 paired episodes for 2 points, now minutes of planning; then a real robot.

Together: world-model planning treated as a systems problem, with the evaluation methodology to show that speed did not cost behaviour.

Take-home

- A JEPA planner at batch 1 is bound by the latency of a long chain of tiny steps, not by FLOPs or bandwidth.
- Sample-efficient planning (iCEM) only pays once execution stops being launch-bound: 0.92x in eager PyTorch, 1.38x in a kernel.
- One persistent kernel per plan: 622 ms -> 46.5 ms on the same H100, same model, same planner, same task success.

Paper, kernel, per-call latency records and per-episode outcomes: docs/paper and megakernel/ in the project repository.